

■ Technical Documentation

DocSentry

An autonomous agent that keeps documentation honest

“The docs defended themselves.”

When code changes in a way that makes the docs lie, DocSentry detects it, opens a GitHub issue (or a fix PR), and verifies its own fix — all driven by a local, zero-credit LLM.

Architecture	PERCEIVE → REASON → ACT → VERIFY
LLM cost	\$0 (local Ollama + local embeddings)
Stack	FastAPI ▪ tree-sitter ▪ ChromaDB ▪ React

Contents

1	What it is & the use case	2
1.1	The problem	2
1.2	The use case	2
1.3	The demo, in one sentence	2
2	How it works (architecture)	2
2.1	Tech stack	2
3	The agentic loop, module by module	3
4	Confidence-based routing	5
5	How it was made	5
5.1	The key pivot: Anthropic → local Ollama	6
5.2	Bugs fixed while building	6
5.3	Build environment notes	6
6	Flow, tokenization & credit optimization	6
6.1	The biggest lever: local models = \$0	6
6.2	Never send code to the model — send deltas	6
6.3	Bound the context with retrieval, don't dump the docs	7
6.4	Hybrid retrieval keeps N small	7
6.5	Gate expensive work behind cheap signals	7
6.6	Tight, structured output	7
6.7	Loop-safety = no wasted runs	7
6.8	Summary table	7
7	Data contracts	7
8	Running it	8
8.1	Prerequisites	8
8.2	One-shot run (CLI)	8
8.3	Live server	9
8.4	Dashboard	9
8.5	Docker	9
9	Security model	9
10	Limitations & known trade-offs	9
11	Future improvements	10
12	FAQ	10

1 What it is & the use case

1.1 The problem

Documentation drifts. A developer flips a default (`safe=True` → `safe=False`), renames a function, or changes a signature — and the README that describes the old behavior silently becomes false. Nobody notices until a user follows the docs and gets burned. Traditional CI can't catch this: the code compiles, the tests pass, but the prose now lies.

1.2 The use case

DocSentry watches a repository. On every push it asks a single question for each code change:

“Does any documentation still make a claim this change just made false?”

If yes, it acts, scaling its authority to its certainty:

- **High confidence** → opens a pull request that patches the doc.
- **Medium confidence** → opens a “Docs Lie” issue for a human to review.
- **Low confidence** → logs and moves on (no noise).

Who it's for

Teams whose docs *are* the product — SDKs, APIs, developer tools, OSS libraries — where a lying README erodes trust faster than a bug. It also doubles as a strong portfolio/learning project for agentic AI, since it implements every stage of the agent loop as a separate, inspectable module.

1.3 The demo, in one sentence

Flip `safe=True` to `safe=False` in the testbed, push, and within seconds a GitHub issue appears, authored by the agent, pointing at the exact README line that is now a lie.

2 How it works (architecture)

DocSentry is a classic **PERCEIVE** → **REASON** → **ACT** → **VERIFY** agent. Each stage below is its own inspectable module.

Key idea

A whitespace or comment change produces zero semantic changes upstream, so the expensive LLM call is never triggered by noise — only meaningful structural changes flow through the pipeline.

2.1 Tech stack

Layer	Choice	Why
Code understanding	tree-sitter (tree-sitter-python)	Real AST parsing — understands structure, not text lines
Git access	GitPython	Before/after blobs for any commit, no shelling out
Retrieval	ChromaDB sentence-transformers (all-MiniLM-L6-v2)	+ Local vector search; no embedding API cost

Layer	Choice	Why
LLM brain	Ollama (orca-mini) via OpenAI-compatible client	Zero API credits, fully local
Actions Server	PyGithub FastAPI + uvicorn	Opens issues/PRs on the target repo Async webhook receiver + dashboard API
Persistence	SQLite (stdlib)	Zero-config run history
Dashboard	React + Vite	Live-polling view of every run
Config	pydantic-settings	Typed .env loading

3 The agentic loop, module by module

core/git_ops.py --- PERCEIVE (raw)

`get_commit_changes(repo, hash)` diffs a commit against its parent and returns a list of `FileChange(path, change_type, before, after)` — the full text of each touched file before and after. This is the only place raw file content is read.

agents/diff_analyzer.py --- PERCEIVE (semantic)

The heart of the “understanding.” Instead of diffing text lines, it:

1. Parses both versions with tree-sitter into ASTs.
2. Extracts a `FunctionSig(name, params, docstring)` snapshot for every function.
3. Compares snapshots and emits `SemanticChange` objects: `function_added`, `function_removed`, `params_changed`, `docstring_changed`.

Why this matters: a whitespace or comment change produces zero semantic changes, so the expensive downstream LLM is never called for noise.

core/parser.py + core/vector_store.py --- REASON (retrieve)

- `parser.py` splits every markdown file into heading-anchored `DocSections` (with line ranges).
- `vector_store.py` embeds each section locally and stores it in ChromaDB. `reindex()` rebuilds the index; `search(query, n)` returns the top-N most similar sections.

agents/doc_linker.py --- REASON (retrieve, hybrid)

`link_change_to_docs(change)` builds a query from the changed function’s name plus a human-readable detail, runs the vector search, then applies an **exact-name keyword boost**: any section that literally mentions the function name is ranked above pure-similarity hits. This hybrid approach (dense embeddings + sparse keyword signal) makes a small n (3) reliable.

agents/divergence.py --- REASON (judge) --- the brain

Sends the change plus one doc section to the local LLM and demands strict JSON (shown below).

Robustness built in:

- `response_format={"type": "json_object"}` forces the model to emit JSON (small models otherwise reply in prose).
- `_parse_verdict()` strips code fences, extracts the first `{...}` block if the model wraps it in text, and coerces types — so a chatty model never crashes the pipeline.
- Uses the OpenAI client pointed at Ollama (`http://localhost:11434/v1`), so it costs nothing.

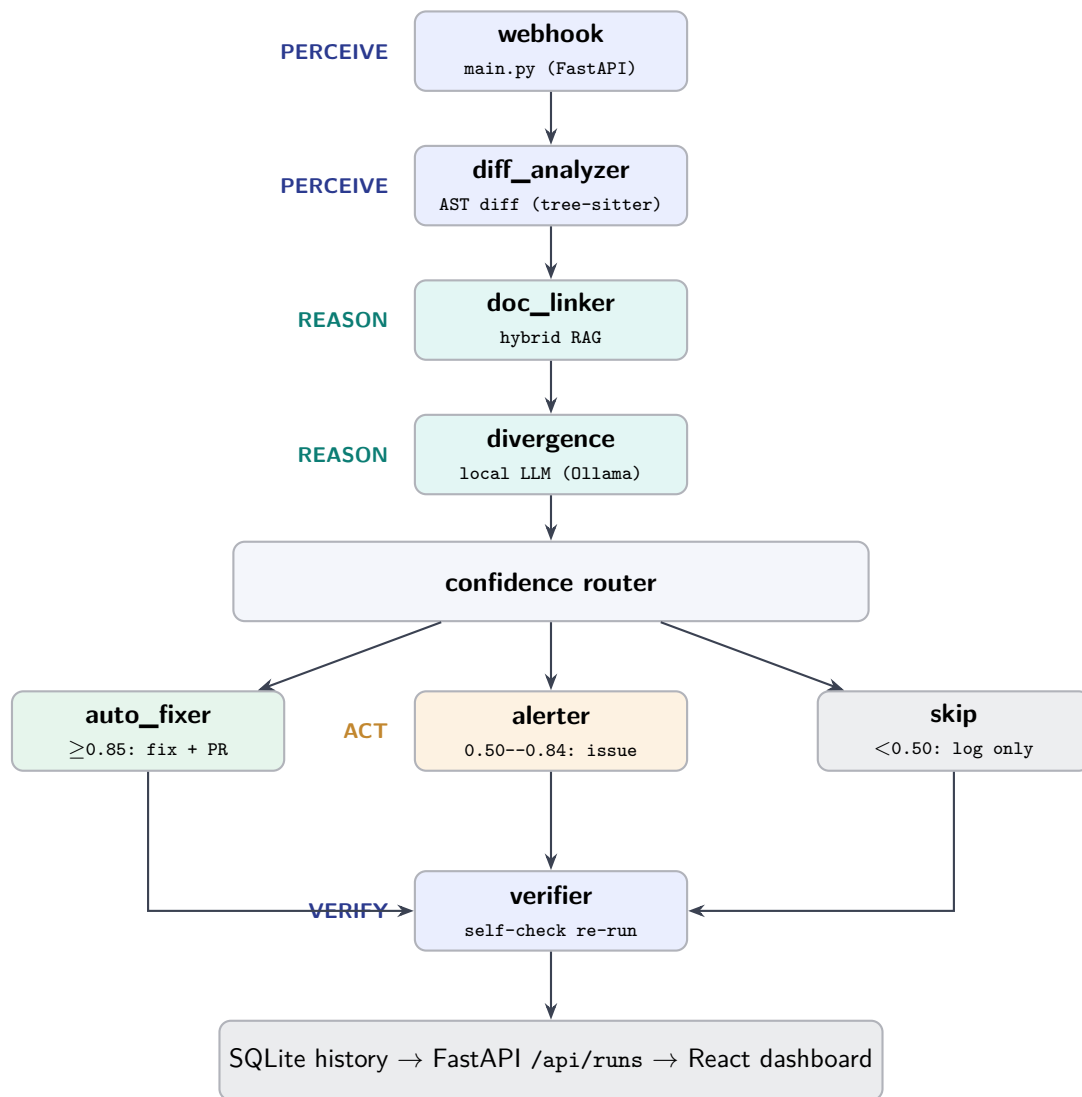


Figure 1: The DocSentry agent loop: webhook intake, semantic diffing, retrieval, LLM judgement, confidence routing, action, and self-verification.

```
{ "diverged": true, "confidence": 0.0-1.0, "mismatch": "...", "suggested_fix": "..." }
```

```
agents/auto_fixer.py --- ACT (autonomous)
```

For high-confidence verdicts: creates a docsentry/fix-<timestamp> branch, replaces the offending doc lines with suggested_fix, commits, pushes, and opens a PR via PyGithub. Returns to the base branch so the working tree stays clean.

```
agents/alerter.py --- ACT (human-in-the-loop)
```

For medium confidence: opens a “Docs Lie” issue with the code change, the suspected lie, the confidence score, and a suggested fix. Labels were intentionally dropped so a fresh repo without predefined labels doesn’t 422.

```
agents/verifier.py --- VERIFY (self-check)
```

Before a fix ships, re-runs `check_divergence` on the patched content. If the doc *still* lies, the auto-fix is downgraded to an issue — the guardrail against LLM overconfidence: the agent grades its own homework before submitting it.

```
pipeline.py --- the orchestrator
```

Wires all of the above into `run_pipeline(commit_hash)` and returns a list of result dicts (`auto_fixed` / `alerted` / `clean` / `low_confidence_skip` / `fix_failed_verification`). Runnable standalone: `python -m docsentry.pipeline`.

```
main.py --- the server
```

FastAPI app exposing:

- POST `/webhook/github` — HMAC-verified push receiver; skips the agent's own fix branches to prevent infinite self-triggering; runs the pipeline as a background task.
- GET `/api/runs` — recent run history from SQLite.
- GET `/api/stats` — aggregate counts for the dashboard tiles.

4 Confidence-based routing

The single most important design idea: **thresholded autonomy**. The agent doesn't act with the same authority on every verdict — it scales its autonomy to its certainty.

Confidence	Action	Rationale
≥ 0.85 (AUTOFIX_THRESHOLD)	✓ Auto-fix → open PR (after self-verify)	Certain enough to propose a concrete change
0.50–0.84 (ALERT_THRESHOLD)	Alert → open issue	Probably wrong, but a human should decide
< 0.50	✗ Log & skip	Too uncertain to be worth anyone's attention

Caution

Both thresholds are tunable in `.env`. With the small local `orca-mini` model, verdicts cap around ~ 0.5 , so everything currently routes to alert — a deliberate, safe default. Lowering `AUTOFIX_THRESHOLD` (or swapping in a stronger local model) unlocks auto-fix PRs.

5 How it was made

Built in six phases, each ending in a runnable checkpoint (test-driven, “always working”):

Phase	Deliverable	Proof of life
0	Env, venv, deps, testbed repo, <code>.env</code>	<code>check_setup.py</code> all-green
1	Diff analyzer (tree-sitter)	Detects <code>divide</code> default flip
2	Code↔doc linker (ChromaDB)	Finds the README <code>divide</code> section, exact match
3	Divergence detector (LLM)	Returns <code>diverged=true</code> verdict

Phase	Deliverable	Proof of life
4	Agentic actions (fix / alert / verify)	Opens a real GitHub issue/PR
5	Webhook server + React dash-board	Push → auto-caught, shown live
6	Dockerfile, CLI, requirements.txt	Containerized, one-command onboarding

5.1 The key pivot: Anthropic → local Ollama

Phase 3 was originally written against the Anthropic API. Mid-build, the API returned 400: "credit balance is too low". Rather than pay, the LLM layer was rewired to a local Ollama model through the OpenAI-compatible client. This kept the exact same JSON contract while dropping the per-token cost to zero — `config.py` no longer requires an Anthropic key.

5.2 Bugs fixed while building

- `core/db.py`: the run-history query read a non-existent `commit` column → corrected to `commit_hash` (would have crashed every dashboard load).
- `agents/alerter.py`: removed `labels=[...]` to avoid a 422 on repos without those labels predefined.
- `agents/divergence.py`: added JSON-mode plus a tolerant parser so a small model's prose replies don't silently register as "clean."

5.3 Build environment notes

The package folder is the project root (flattened layout), so `import docsentry` needs the parent on `sys.path`, while relative paths (`../docsentry-testbed`, `./chroma_db`, `docsentry.db`) assume the CWD is the `docsentry/` folder. The canonical launch sets `PYTHONPATH=..` from inside `docsentry/`.

6 Flow, tokenization & credit optimization

This is where DocSentry is deliberately frugal. Credit/token usage was optimized at every stage of the pipeline, not just by picking a cheap model.

6.1 The biggest lever: local models = \$0

- LLM inference runs on Ollama locally — no per-token API billing at all.
- Embeddings run on `all-MiniLM-L6-v2` locally via `sentence-transformers` — no embedding API.
- Net external LLM/embedding spend: **zero**. The only "budget" is your own CPU/RAM.

6.2 Never send code to the model — send deltas

The LLM never sees a source file. `diff_analyzer` distills a change into a single human-readable line like:

```
`divide` signature changed: (a, b, safe=True) -> (a, b, safe=False)
```

That one string (a few dozen tokens) replaces what would otherwise be hundreds of tokens of raw diff. Structural understanding happens in `tree-sitter` (free), so the model only reasons about the meaning, not the text.

6.3 Bound the context with retrieval, don't dump the docs

Instead of sending the entire documentation to the model, `doc_linker` retrieves only the top-3 most relevant sections, and the model judges one section at a time. Context per call stays tiny and constant regardless of how large the docs grow.

6.4 Hybrid retrieval keeps N small

The exact-name keyword boost means the truly relevant section reliably lands in the top few results, so $n = 3$ is enough. Fewer candidate sections mean fewer LLM calls per change.

6.5 Gate expensive work behind cheap signals

- **AST pre-filter:** whitespace/comment/formatting changes produce zero `SemanticChanges`, so the LLM is never invoked for them. Non-`.py` files are skipped outright.
- **Per-commit scoping:** only files touched by the commit are analyzed — never a full-repo scan.
- **Confidence routing:** the extra self-verify LLM call only fires for high-confidence auto-fix candidates. The common case (an alert or a clean verdict) costs exactly one LLM call.

6.6 Tight, structured output

- `max_tokens=800` caps every response.
- `response_format=json_object` eliminates wasted “Here is the JSON you asked for...” preamble tokens and prevents costly retries from unparseable prose.

6.7 Loop-safety = no wasted runs

The webhook skips pushes on the agent's own `docsentry/fix-*` branches, so an auto-fix PR never retriggers the whole pipeline on itself — preventing an infinite (and expensive) feedback loop.

6.8 Summary table

Technique	Where	Saves
Local Ollama LLM	<code>divergence.py</code>	100% of LLM API cost
Local embeddings	<code>vector_store.py</code>	100% of embedding API cost
AST delta, not raw code	<code>diff_analyzer.py</code>	Majority of prompt tokens
Top-N section retrieval	<code>doc_linker.py</code>	Unbounded → constant context
AST pre-filter / <code>.py</code> skip	<code>diff_analyzer.py</code>	Whole LLM calls on noise
Per-commit scoping	<code>git_ops.py</code>	Full-repo re-analysis
Verify only on high-conf	<code>pipeline.py</code>	Redundant second call
<code>max_tokens</code> + JSON mode	<code>divergence.py</code>	Output tokens + retries
Self-branch skip guard	<code>main.py</code>	Infinite self-triggering

7 Data contracts

The modules are decoupled by plain dicts, which keeps each stage independently testable.

► SemanticChange (from diff_analyzer)

```
{ "file": "calculator.py", "kind": "params_changed",
  "name": "divide", "detail": "`divide` signature changed: (...) -> (...)" }
```

► Doc hit (from doc_linker / vector_store.search)

```
{ "id": "README.md:7", "content": "...", "distance": 0.31,
  "meta": { "file": "README.md", "heading": "divide(a, b, safe=True)",
            "start_line": 7, "end_line": 11},
  "exact_match": true }
```

► Verdict (from divergence)

```
{ "diverged": true, "confidence": 0.5,
  "mismatch": "doc claims safe defaults to True, but it now defaults to False",
  "suggested_fix": "...", "doc_id": "...", "doc_file": "...", "doc_heading": "..." }
```

► Run result (from pipeline, stored in SQLite, served to the dashboard)

```
{ "status": "alerted", "issue": "https://github.com/.../issues/1",
  "change": "`divide` signature changed: (...) -> (...)" }
```

8 Running it

8.1 Prerequisites

- Python 3.11+ with the project .venv and deps installed
- Ollama running locally with a model pulled: `ollama pull orca-mini`
- Node.js 20+ (only for the dashboard)
- A .env with GITHUB_TOKEN, TARGET_REPO, LOCAL_REPO_PATH, WEBHOOK_SECRET, thresholds

8.2 One-shot run (CLI)

```
cd docsentry
$env:PYTHONPATH = ".."
.venv\Scripts\python -m docsentry.cli init // index the docs
.venv\Scripts\python -m docsentry.pipeline // run the agent on the latest commit
```

8.3 Live server

```
cd docsentry
$env:PYTHONPATH = ".."
.venv\Scripts\python -m uvicorn docsentry.main:app --port 8000
```

Then expose it (ngrok http 8000) and register a GitHub webhook → <tunnel>/webhook/github (content type application/json, secret = WEBHOOK_SECRET, “just the push event”).

8.4 Dashboard

```
cd docsentry\dashboard
npm install
npm run dev // http://localhost:5173
```

8.5 Docker

```
docker build -t docsentry .
docker run --env-file .env -p 8000:8000 -v ${PWD}\..\docsentry-testbed:/repo docsentry
```

9 Security model

- **Webhook authenticity:** every payload is HMAC-SHA256 verified against WEBHOOK_SECRET using hmac.compare_digest (constant-time). Unsigned or mismatched requests get 401 before any work.
- **Least-privilege token:** the GitHub PAT is fine-grained, scoped to the single target repo, with only Contents/Issues/PRs/Webhooks write.
- **Secrets never committed:** .env, chroma_db/, and docsentry.db are gitignored.
- **No data leaves the machine:** because the LLM and embeddings are local, source and docs are never sent to a third-party API — a real privacy/compliance advantage over cloud-LLM approaches.

10 Limitations & known trade-offs

- **Small-model ceiling:** orca-mini caps confidence ~0.5, so auto-fix PRs don't trigger under the default 0.85 threshold — everything becomes an alert. A stronger local model (or lower threshold) changes this.
- **Multi-alert noise:** the pipeline judges the top-3 linked sections independently, so one code change can raise more than one issue. Acting only on the single best match would reduce noise.
- **Python-only:** diff_analyzer currently parses Python. Other languages need their own tree-sitter grammar plus an extract_functions mirror.
- **Markdown-only docs:** parser.py handles .md; other doc formats aren't indexed.
- **Function-level granularity:** it reasons about signatures/docstrings, not deep behavioral changes inside a function body.
- **Launch sensitivity:** relative paths assume a specific working directory (see §5).

11 Future improvements

- Multi-language: add `tree-sitter-javascript/-typescript` and `mirror extract_functions`.
- Single-best-match mode to cut alert noise.
- Absolute path anchoring in `config.py` so it runs from any directory.
- Docs Health Score badge (shields.io endpoint returning % clean).
- PR-comment mode: comment on the offending PR instead of opening a new issue.
- Slack/Discord alerts via a webhook.
- Dogfooding: point DocSentry at its own repo.
- Model routing: small local model for triage, escalate only uncertain cases to a stronger model.

12 FAQ

Does DocSentry cost anything to run?

No ongoing API cost. The LLM (Ollama) and the embedding model (`all-MiniLM-L6-v2`) both run locally, so the only “budget” consumed is your own CPU/RAM — see §6.1.

Does source code ever leave my machine?

No. Both the LLM and the embeddings run locally, so code and docs are never sent to a third-party API — see §9.

What happens if the local LLM replies with prose instead of JSON?

`divergence.py` forces JSON mode and runs the reply through a tolerant parser that strips code fences and extracts the first JSON object, so a chatty small model never crashes the pipeline — see §3.

Why doesn't auto-fix ever trigger by default?

The default local model, `orca-mini`, caps its confidence around 0.5, below the 0.85 `AUTOFIX_THRESHOLD`. Everything currently routes to an alert instead — a deliberate, safe default. Lower the threshold or swap in a stronger local model to unlock auto-fix PRs — see §4 and §10.

Can an auto-fix PR trigger DocSentry on itself?

No. The webhook explicitly skips pushes on the agent's own `docsentry/fix-*` branches, preventing an infinite feedback loop — see §6.7.

DocSentry — *because a lying README is worse than no README.*